

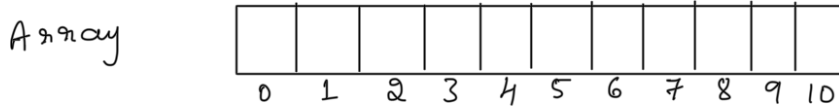
Array List In JAVA

DAY 67
15 / 09 / 2025

Why ArrayList ?

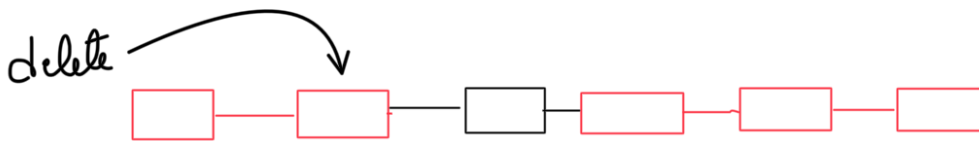
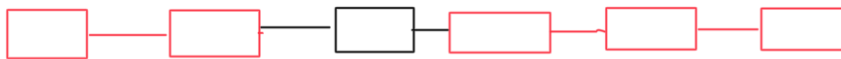
As I discussed Earlier Array Have Some drawback

In Array We have Contiguous Space of Memory to store the Elements and it Will Have Index's



Deleting an Element In the array or Inserting an Element In the array takes $O(n)$ shifting operations In Worst Case

In Case of linked List

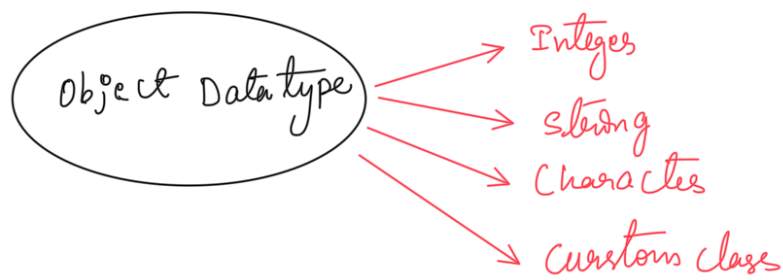


Insertion
And
deletion

do not require
any shifting
operations like
How we used
to do in Array

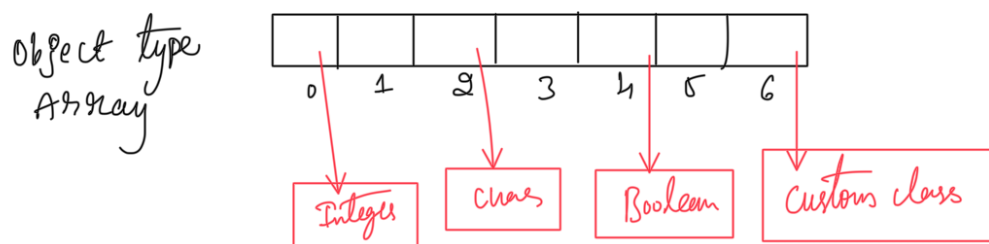
Combining the goodness of Array and goodness of list
Neha data structure is created which is called
ArrayList

Earlier In Java ArrayList Work for Any Data type



When the ArrayList was created it was declared as Object data type which can store the values of any data type by storing the different object types address in the index. Basically it's a Heterogeneous

But array is Homogeneous



Later in the year 2004 generics were introduced then the ArrayList is upgraded

Now ArrayList for specific data type can be created

While declaring the array itself we can declare its data type

`ArrayList < specific Data type > name = new ArrayList < > ();`

It may be stores
Elements of unique
type

Integer
Character
String
Custom class

ArrayList Have Methods for adding, deleting, and perform some other operations or tasks

Java ArrayList Methods Cheat Sheet			
Category	Method	Description	Example
Add	<code>add(E e)</code>	Adds element at end	<code>list.add("Apple")</code>
	<code>add(int index, E e)</code>	Inserts at specific index	<code>list.add(1, "Mango")</code>
	<code>addAll(Collection c)</code>	Adds all elements from another collection	<code>list.addAll(otherList)</code>
Access / Update	<code>get(int index)</code>	Returns element at index	<code>list.get(0)</code>
	<code>set(int index, E e)</code>	Replaces element at index	<code>list.set(1, "Banana")</code>
Remove	<code>remove(int index)</code>	Removes element at index	<code>list.remove(0)</code>
	<code>remove(Object o)</code>	Removes first occurrence of object	<code>list.remove("Apple")</code>
	<code>removeAll(Collection c)</code>	Removes all from given collection	<code>list.removeAll(otherList)</code>
	<code>clear()</code>	Removes all elements	<code>list.clear()</code>
Search	<code>contains(Object o)</code>	Checks if element exists	<code>list.contains("Apple")</code>
	<code>indexOf(Object o)</code>	Returns first index of element	<code>list.indexOf("Mango")</code>
	<code>lastIndexOf(Object o)</code>	Returns last index of element	<code>list.lastIndexOf("Apple")</code>
Info	<code>size()</code>	Number of elements	<code>list.size()</code>
	<code>isEmpty()</code>	Checks if list is empty	<code>list.isEmpty()</code>
Conversion	<code>toArray()</code>	Converts to array	<code>Object[] arr = list.toArray()</code>
Iteration	<code>iterator()</code>	Returns iterator	<code>Iterator it = list.iterator()</code>
	(Enhanced for loop)	Loop over elements	<code>for(String s : list)</code>
Bulk Ops	<code>retainAll(Collection c)</code>	Keeps only common elements	<code>list.retainAll(otherList)</code>
Utility (Collections class)	<code>Collections.sort(list)</code>	Sorts list	
	<code>Collections.reverse(list)</code>	Reverses order	
	<code>Collections.shuffle(list)</code>	Random shuffle	

The above table contains some methods of ArrayList which performs the certain operations.

Operation	Best Case	Worst Case	Why
<code>add(E e) (end)</code>	$O(1)$	$O(n)$	Resize & copy when full
<code>add(int index, E e)</code>	$O(n)$	$O(n)$	Shifting elements
<code>get(int index)</code>	$O(1)$	$O(1)$	Direct array access
<code>set(int index, E e)</code>	$O(1)$	$O(1)$	Direct array access
<code>remove(int index)</code>	$O(n)$	$O(n)$	Shifting elements
<code>remove(Object o)</code>	$O(n)$	$O(n)$	Search + shifting

Time complexity table

Internal Working of ArrayList In JAVA

Case ① Creation

`ArrayList<Integer> list = new ArrayList<>();`

Internally creates a `Object[]` elementData type array.

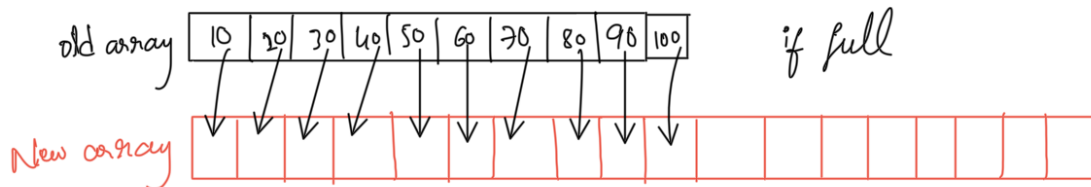
Default Capacity = 10 (if not specified)
keeps a variable Size (number of elements currently stored)



Case ② Adding elements

a) At End (`add(E e)`)

- places element at `elementData[size]`
- Increment `Size`
- if Array is full $\rightarrow$ creates new array ($1.5 \times \text{old size}$) and copies old elements



Time Complexity $O(1)$ But $O(n)$ if resize happens

b) At index (`add(int index, E e)`)

- shifts all elements from that index to the right
- Insert element at given position

Time Complexity $O(n)$

Case 3 Accessing elements

```
list.get(i) ;  
list.set(i, value) ;
```

Time complexity $O(1)$

Directly uses array
Indexing $\rightarrow$ array[i]

Case 4 Removing elements

a) By Index

```
list.remove(i) ;
```

- Removes element at index
- shifts all elements after it one step left
- Decrement size

Time complexity $O(n)$

b) By Value

```
list.remove("Apple") ;
```

- First searches for element $\rightarrow$ $O(n)$
- Then shifts $\rightarrow$ $O(n)$

Time complexity $O(n)$

Case 5 Searching

```
list.contains(x)  
list.indexOf(x)
```

Time complexity $O(n)$

Linear search through elements



Case 6 utility operations

- `Size()` → returns current number of elements $O(1)$
- `isEmpty()` → check if list has zero size $O(1)$
- `clear()` → Sets size = 0 (array still exists)

⚡ Summary of Internal Working

- **Data Structure:** Backed by a resizable array (`Object[]`).
- **Capacity Handling:** Starts with 10, grows by 1.5x when full.
- **Insert:**
 - At end → $O(1)$ amortized.
 - At index → $O(n)$ (shifting).
- **Access/Update** → $O(1)$.
- **Remove:** $O(n)$ (shifting).
- **Search:** $O(n)$.

🎯 Key Idea

👉 ArrayList is best for random access and append operations, but not good for frequent insertions/deletions in the middle.

⚙️ Features of Array used in ArrayList

1. Contiguous memory allocation

- Like an array, ArrayList stores elements in a **continuous block of memory** (`Object[]`).

2. Index-based access

- You can do `list.get(i)` in $O(1)$ time because it's just array indexing (`elementData[i]`).

3. Fast traversal

- Iterating is as fast as looping through a normal array.

features of Array used
In Array list creation

features of list used in
array list →

⚙️ Features of List (Dynamic Collection) used in ArrayList

1. Dynamic resizing

- Unlike arrays (fixed size), ArrayList grows automatically (1.5x rule) when full.

2. Flexible operations

- You can add, remove, search, insert → operations that normal arrays don't directly support

3. Implements List interface

- Provides all list behaviors (add, remove, contains, etc.)
- Maintains **insertion order**.
- Allows **duplicates** and **nulls**.

🔪 So, in short:

- From Array → fast random access ($O(1)$) + contiguous storage.
- From List → dynamic size + rich set of operations.

👉 That's why ArrayList is called a resizable array implementation of the List interface.

Arraylist without specifying any data type (supports multiple datatypes)

```
DSA > ArrayList > J ArrayListDemo.java > Java Language Support > ArrayListDemo > hetArrayList

3 import java.util.ArrayList;
4
5 public class ArrayListDemo {
6
7     // Hetrogenous ArrayList
8     public static void hetArrayList(){
9
10        System.out.println(x:"Array Which Supports All type of data");
11
12        ArrayList list = new ArrayList<>();
13
14        list.add(e:"Veeresh"); // i added String Class Type Data
15
16        list.add(e:68); // i added Integer class type Data
17
18        //Type Casting Required to access (Integer -- int)
19        int number = (int) list.get(index:1);
20
21        System.out.println("Value at Index 1 after typeCasting "+number);
22
23        //Type Casting
24        String name = (String) list.get(index:0);
25
26        System.out.println("Value at Index 0 after typeCasting "+name);
27
28        System.out.println("Size Of the List: "+ list.size());
29    }
30
31    Run | Debug | Run main | Debug main
32    public static void main(String[] args) {
33
34        hetArrayList();
35    }
36 }
```

Debug: ArrayListDemo

```
(eval):1: no such file or directory: upgrade.
/bin
(eval):5: no such file or directory: upgrade.
/bin
(eval):5: no such file or directory: upgrade.
/bin

/usr/bin/env /Users/kveeresh/Library/Java/JavaVirtualMachine
s/openjdk-24.0.1/Contents/Home/bin/java -agentlib:jdwp=transp
ort=dt_socket,server=n,suspend=y,address=localhost:55083 --en
able-preview -XX:+ShowCodeDetailsInExceptionMessages -cp /Use
rs/kveeresh/Library/Application\ Support/Code/User/workspace5
storage/2ecbf2b3509965b320b049bd629e6946/redhat.java/jdt_ws/Co
mputer-Science_4e28d2eb/bin DSA.ArrayList.ArrayListDemo
Could not find a command named "sdk-path".

Run 'flutter -h' (or 'flutter <command> -h') for available fl
utter commands and options.
(base) kveeresh@KS-MacBook-Air-2 Computer-Science % /usr/bin
/env /Users/kveeresh/Library/Java/JavaVirtualMachines/openjdk
-24.0.1/Contents/Home/bin/java -agentlib:jdwp=transport=dt_so
cket,server=n,suspend=y,address=localhost:55083 --enable-prev
iew -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/kveere
sh/Library/Application\ Support/Code/User/workspaceStorage/2e
cbf2b3509965b320b049bd629e6946/redhat.java/jdt_ws/Computer-Sc
ience_4e28d2eb/bin DSA.ArrayList.ArrayListDemo
Array Which Supports All type of data
Value at Index 1 after typeCasting 68
Value at Index 0 after typeCasting Veeresh
Size Of the List: 2
(base) kveeresh@KS-MacBook-Air-2 Computer-Science %
```

Type Casting

Arraylist with generics (store specified data type)

```
DSA > ArrayList > J ArrayListDemo.java > Java Language Support > ArrayListDemo > arrayListWithGe

5 public class ArrayListDemo {
6
7     // ArrayList With Generics
8     public static void arrayListWithGenerics(){
9
10        System.out.println(x:"Array Which Supports single type of data");
11
12        System.out.println(x:"Taking String data Type as Example");
13        ArrayList<String> list = new ArrayList<>();
14
15        list.add(e:"Veeresh"); // added String Class Type Data
16
17        list.add(e:"Namaskara My LinkedIn Family");
18
19        //Type Casting Required to access
20        String number = (String) list.get(index:1);
21
22        System.out.println("Value at Index 1 after typeCasting: "+number);
23
24        //Type Casting
25        String name = (String) list.get(index:0);
26
27        System.out.println("Value at Index 0 after typeCasting: "+name);
28
29        System.out.println("Size Of the List: "+ list.size());
30    }
31
32    Run | Debug | Run main | Debug main
33    public static void main(String[] args) {
34
35        //hetArrayList();
36
37        arrayListWithGenerics();
38    }
39 }
```

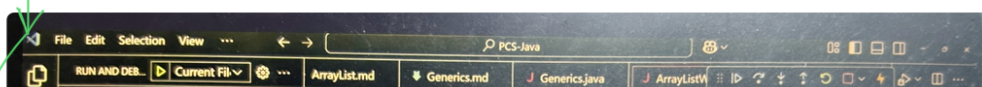
Debug: ArrayListDemo

```
(eval):1: no such file or directory: upgrade.
/bin
(eval):5: no such file or directory: upgrade.
/bin
(eval):5: no such file or directory: upgrade.
/bin

/usr/bin/env /Users/kveeresh/Library/Java/JavaVirtualMachine
s/openjdk-24.0.1/Contents/Home/bin/java -agentlib:jdwp=transp
ort=dt_socket,server=n,suspend=y,address=localhost:55186 --en
able-preview -XX:+ShowCodeDetailsInExceptionMessages -cp /Use
rs/kveeresh/Library/Application\ Support/Code/User/workspace5
storage/2ecbf2b3509965b320b049bd629e6946/redhat.java/jdt_ws/Co
mputer-Science_4e28d2eb/bin DSA.ArrayList.ArrayListDemo
Could not find a command named "sdk-path".

Run 'flutter -h' (or 'flutter <command> -h') for available fl
utter commands and options.
(base) kveeresh@KS-MacBook-Air-2 Computer-Science % /usr/bin
/env /Users/kveeresh/Library/Java/JavaVirtualMachines/openjdk
-24.0.1/Contents/Home/bin/java -agentlib:jdwp=transport=dt_so
cket,server=n,suspend=y,address=localhost:55186 --enable-prev
iew -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/kveere
sh/Library/Application\ Support/Code/User/workspaceStorage/2e
cbf2b3509965b320b049bd629e6946/redhat.java/jdt_ws/Computer-Sc
ience_4e28d2eb/bin DSA.ArrayList.ArrayListDemo
Array Which Supports single type of data
Taking String data type as Example
Value at Index 1 after typeCasting: Namaskara My LinkedIn Fam
ily
Value at Index 0 after typeCasting: Veeresh
Size Of the List: 2
(base) kveeresh@KS-MacBook-Air-2 Computer-Science %
```

No Need of this type Casting



Handwritten notes:

- Creating arraylist without specifying any data type
- Creating arraylist of object type

```

1 import java.util.ArrayList;
2
3 public class ArrayListWorkshop {
4
5     public static void demoArrayList()
6     {
7         ArrayList list = new ArrayList(); list = ArrayList<String> list = ArrayList<String>
8         list.add("mahesh"); // zero index string list = ArrayList<String>
9     }
10 }
  
```

Go to Super Implementation

boolean java.util.ArrayList.add(Object e)

Appends the specified element to the end of this list.

Specified by: add(...) in List, Overrides: add(...) in AbstractList

Parameters:

- element to be appended to this list

Returns:

- true (as specified by Collection.add)

Handwritten notes:

- returns the reference of ArrayList of type String
- returns the object reference
- Bcz they are created in Heap Memory

```

26 public static ArrayList<String> GetNames()
27 {
28     ArrayList<String> names = new ArrayList<>();
29     names.add("mahesh");
30     names.add("sangeeta");
31     names.add("amit");
32     names.add("santosh");
33     names.add("mahesh");
34     names.add("mahat");
35
36     return names;
37 }
  
```

Handwritten notes:

- Printing arraylist

```

55 public class ArrayListDemo {
56
57     public static void printArrayList()
58     {
59         System.out.println();
60         System.out.println("Printing ArrayList");
61         // Create ArrayList of Strings
62         ArrayList<String> names = new ArrayList<>();
63
64         // Add names
65         names.add("Veeresh");
66         names.add("Karan");
67         names.add("Pranav");
68         names.add("Sachin");
69         names.add("Rudresh");
70
71         // Printing using for-each loop
72         System.out.println("Names in the ArrayList:");
73         for (String name : names) {
74             System.out.println(name);
75         }
76
77         // Printing directly (ArrayList has toString())
78         System.out.println("Using ArrayList toString():");
79         System.out.println(names);
80
81         System.out.println();
82
83         public static void main(String[] args) {
84             //hetArrayList();
85             // arraylistWithGenerics();
86             printArrayList();
87         }
88     }
89 }
  
```

Terminal Output:

```

(eval):1: no such file or directory: upgrade.
(eval):5: no such file or directory: upgrade.
(eval):5: no such file or directory: upgrade.
/usr/bin/env: /Users/kveeresh/Library/Java/JavaVirtualMachine
s/openjdk-24.0.1/Contents/Home/bin/java -agentlib:jdwp=transp
ort=dt_socket,server=n,suspend=y,address=localhost:55255 --en
able-preview -XX:+ShowCodeDetailsInExceptionMessages -cp /Use
rs/kveeresh/Library/Application\ Support/Code/User/workspace5
storage/2ecbf2b3509965b320b049bd629e6946/redhat.java/jdt_ws/Co
mputer-Science_4e28d2eb/bin DSA.ArrayList.ArrayListDemo
Could not find a command named "sdk-path".

Run 'flutter -h' (or 'flutter <command> -h') for available fl
utter commands and options.
(base) kveeresh@Ks-MacBook-Air-2 Computer-Science % /usr/bin
/env /Users/kveeresh/Library/Java/JavaVirtualMachine/openjdk
-24.0.1/Contents/Home/bin/java -agentlib:jdwp=transport=dt_so
cket,server=n,suspend=y,address=localhost:55255 --enable-prev
iew -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/kveere
sh/Library/Application\ Support/Code/User/workspace5Storage/2e
cbf2b3509965b320b049bd629e6946/redhat.java/jdt_ws/Computer-Sc
ience_4e28d2eb/bin DSA.ArrayList.ArrayListDemo

Printing ArrayList
Names in the ArrayList:
Veeresh
Karan
Pranav
Sachin
Rudresh

Using ArrayList toString():
[Veeresh, Karan, Pranav, Sachin, Rudresh]
  
```


Tomorrow I will Write Arraylist generic datatype code from Scratch, I will implement Some common methods Adding, deleting, printing, Searching, Some utility operations.

Thank you My linkedIn Family See you tomorrow.